
Assignment 2 — Multi-Stage Recommendation: Rating Prediction

Course: ECS 172

Author: Ruhani Rekhi

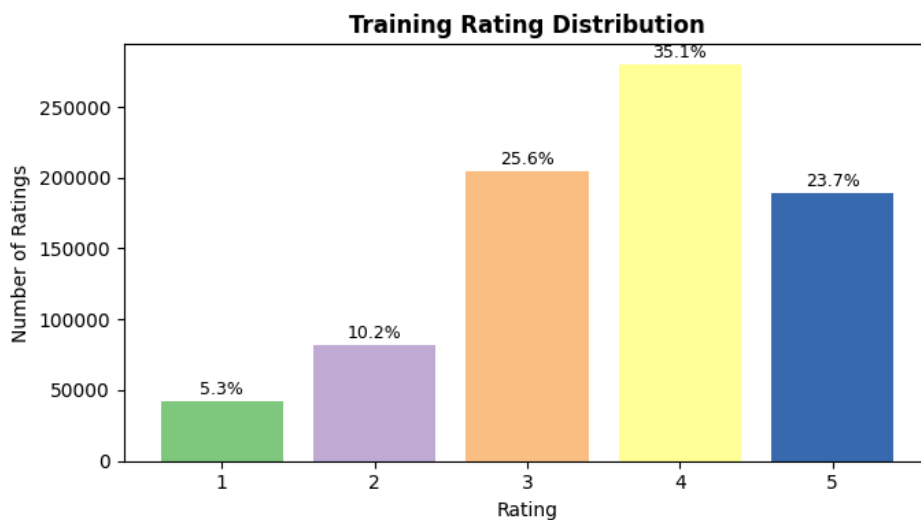
Final submission: submission.csv (genre-personalized hybrid item-item CF, validation RMSE 0.8750)

Table of Contents

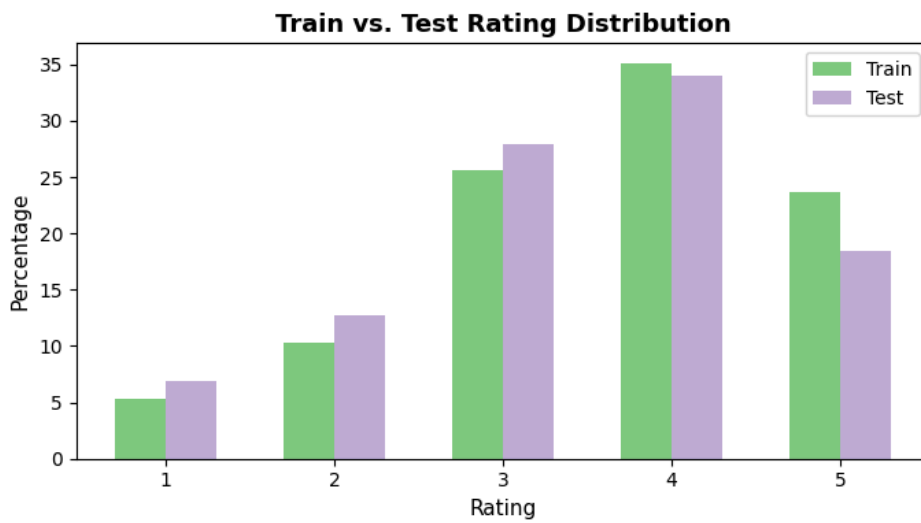
- [Assignment 2 — Multi-Stage Recommendation: Rating Prediction](#)
 - [Table of Contents](#)
 - [Data Exploration](#)
 - [Validation Methodology](#)
 - [Methods](#)
 - [Method 1: Global Mean Baseline](#)
 - [Method 2: User Mean and Item Mean](#)
 - [Method 3: Bias Model](#)
 - [Method 4: Item-Based Collaborative Filtering with Bias Baseline](#)
 - [Method 5: Item-Based Collaborative Filtering with Content Based Features](#)
 - [Result Summary Table](#)
 - [Multi-Stage Discussion: Retrieval & Ranking](#)
 - [Ending Analysis](#)

Data Exploration

We do a full data exploration on the dataset in `data_exploration.ipynb`. We see that the rating distribution has a left skew. The plot shown below demonstrates that almost 60% of the reviews in the training set are 4s and 5s.

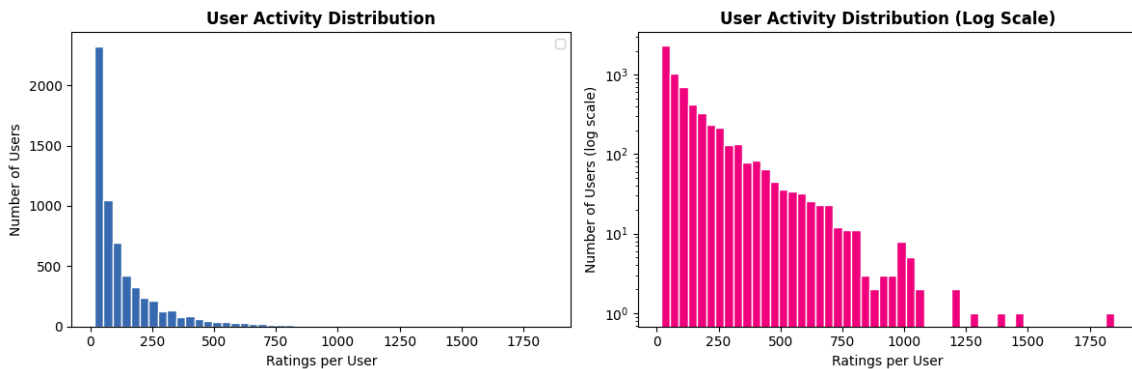


We also examined in the data exploration notebook the train and test split provided by the `split.json` that gave us basic stats of the actual ratings in the test file. Through the distribution below we can see that the train and test ratings have approximately the same rating distribution.



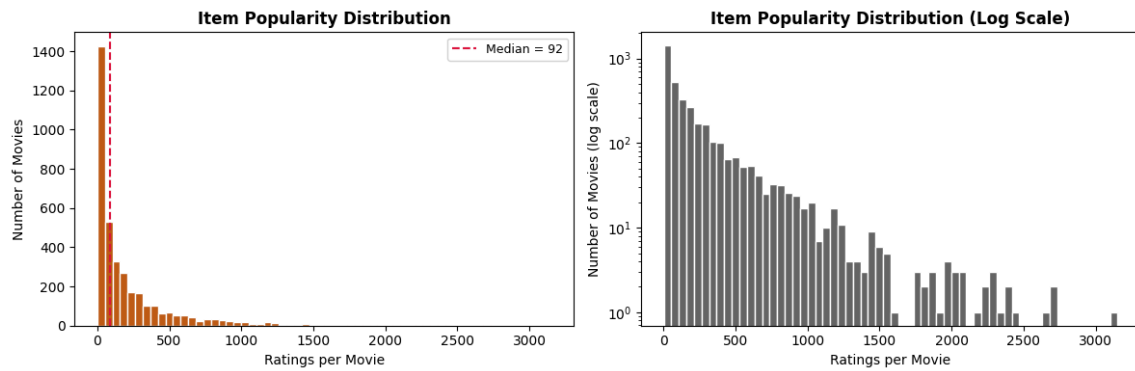
In the data exploration notebook we also investigated how many ratings each user gives to get an idea of if there are some users that make up a hefty portion of all the reviews in the dataset. Through our analysis detailed below we can tell most users have between 50 to 200 ratings but a small outlier group has more than 1000 reviews. Overall, these results seem reasonable and do not cause concern.

Metric	Value
Median ratings per user	76.0
Mean ratings per user	132.08
Minimum ratings per user	16
Maximum ratings per user	1851
Users with less than 20 ratings	416
Users with more than 200 ratings	1,225



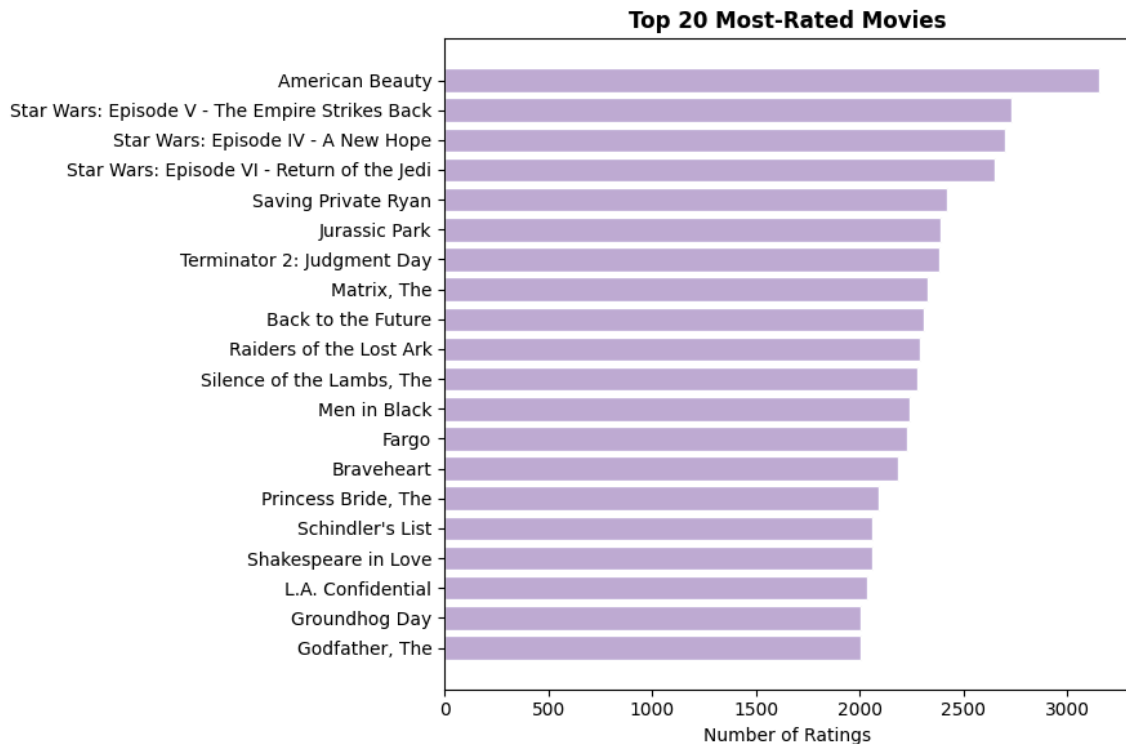
We also look at the item popularity distribution to understand if there is a skew which most likely would be derived from the fact that only blockbusters make up most of the ratings. We see in the table and visualization below that there are some blockbuster films that have quite a lot of ratings and there are also quite a lot of films that have below 10 ratings. This is something to keep in mind for collaborative filtering as there will be a good amount of cold items.

Metric	Value
Median ratings per item	91.5
Mean ratings per item	217.61
Minimum ratings per item	1
Maximum ratings per item	3153
Items with less than 10 ratings	514
Items with over 500 ratings	477



Based on the info provided in the README.pdf we already know there are 40 cold start items in the test set that we don't have any recorded interactions of within the training set.

Here is also a quick visualization of the top most rated movies to get an idea of what we are working with.



Validation Methodology

For validation I used a per-user temporal leave the last 20% split. The logic is as follows: for each user we sort ratings by timestamp, take the oldest 80% as the training data, and use the newest 20% as the validation. This leaves us with the training.csv split into two dataframes, one for training (`training_set`) with 635,723 observations and one for validation (`validation_set`) with 162,035 observations. The hyperparameters for the methods used were tuned based on RMSE evaluation on the validation set.

Methods

I implemented 6 methods of increasing complexity and quality that build upon each other to cover what the previous method could not capture.

Method 1: Global Mean Baseline

This is the simplest method of the bunch where every pair is given the same rating prediction of the global mean of the train ratings (the average of all the ratings in the user-item interaction plot). This is the baseline and we obviously want each of our methods to progressively get less and less error. This method yields a result of...

Validation RMSE: 1.120

Method 2: User Mean and Item Mean

Following the global mean we use the user mean and item mean. The user mean entails predicting each user's rating on the test set movie pair to be the average of the ratings that they have given to the movies they've watched in the training set. This method yields a result of...

User Mean Validation RMSE: 1.0466

We then try utilizing the item mean which predicts the user's rating for a test set movie to be the average rating the test set movie received in the training set (for cold-start items the fallback was the global mean). This method yields a result of...

Item Mean Validation RMSE: 0.9852~

Method 3: Bias Model

This is the combined bias model that combines aspects of the user signal and item quality, using regularization. The general formula is $\mu + \text{user bias} + \text{item bias}$, where user bias tells us how much the user tends to rate above or below mean, and item bias tells us how much some movie is rated above or below mean. We will also use regularization to account for the case where a movie only has 1 rating of 5 stars, which gives a high item bias that dominates. To account for this when calculating the item bias we add some lambda in the denominator so our formula becomes $b_i = \frac{\sum \text{residuals}}{\text{count}} \rightarrow b_i = \frac{\sum \text{residuals}}{\text{count} + \lambda_i}$ (same change respectively for user bias) so in cases where rating count for an item is large the lambda is negligible and if review count is low the lambda shrinks the bias and the predicted rating falls around the global mean.

For this method we have to tune the λ values so we run a simple grid search for λ_i (1, 5, 10, 25, 50) and λ_u (1, 5, 10, 15, 25). The best combination was $\lambda_u = 5$ and $\lambda_i = 1$. This method yields a result of...

Bias Model Validation RMSE: 0.912953

Method 4: Item-Based Collaborative Filtering with Bias Baseline

This is a hybrid recommendation system that uses a baseline bias model defined above and adds neighborhood based collaborative filtering with cosine similarity utilized on the user-item utility matrix.

Main formula:

$$\hat{r}(u, i) = \mu + b_u + b_i + \frac{\sum_j \text{sim}(i, j) \times (r(u, j) - \mu - b_u - b_j)}{\sum_j |\text{sim}(i, j)|}$$

where...

- μ = global mean rating
- b_u = user bias
- b_i = item bias
- b_j = neighbor item bias
- $\text{sim}(i, j)$ = cosine similarity between item i (target) and item j (neighbor)
- $r(u, j)$ = actual rating user u gave to neighbor j

We find the neighbor movies (target item i, only look at movies j that user u has already rated and have positive cosine similarity to i, get top K items). For each neighbor we calculate the residual which is the actual rating subtracted by the baseline prediction (calculated using the bias formula from Method 3).

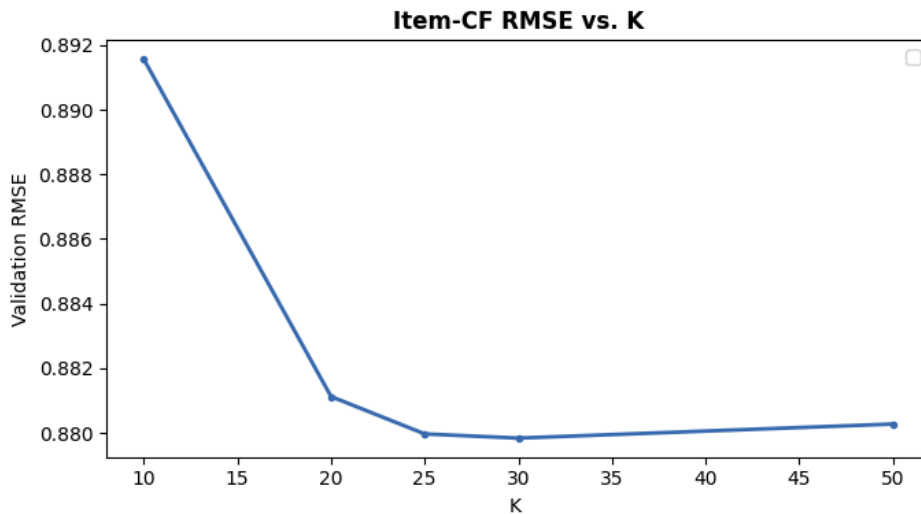
Ex.

- $\mu = 3.5$ (average rating overall)
- $b_u = +0.3$ (Alice rates 0.3 higher than average)
- $b_j = +0.4$ (Movie Inception rating is 0.4 above average)
- Baseline prediction = $3.5 + 0.3 + 0.4 = 4.2$
- Actual rating Alice gave = 4.5
- Residual = $4.5 - 4.2 = +0.3$

We then take a similarity weighted average of the residuals so we can get a number that quantifies how much a user over/under rates movies similar to the target movie i. The final prediction is the baseline bias model plus the collaborative filtering correction we produce by getting the weighted residuals of the neighboring movies. Collaborative filtering essentially is an adjustment to the baseline bias model and adds on to it.

For this method we tuned K, the number of neighbors (10, 20, 25, 30, 50), and evaluated performance through validation RMSE. This method yields a result of...

K=30 Item CF Validation RMSE: 0.8798



Method 5: Item-Based Collaborative Filtering with Content Based Features

This model builds upon Method 4 by adding genre preferences and decade trends into consideration. We use the same collaborative filtering neighbor logic but now utilize a hybrid recommender system approach by combining collaborative filtering with content-based features, in this case genre and decade.

Original baseline:

$$baseline(u, i) = \mu + b_u + b_i$$

New bias equation:

$$baseline(u, i) = \mu + b_u + b_i + \text{genre signal}(u, i) + b_{decade}[\text{decade}(i)]$$

The genre signal tells us how much each user likes each of 18 genres and then averages these affinities across the genres of the target movie (learned through training). For example if the target movie has genres 'action' and 'sci-fi', the signal is the average of the weights for each genre that tell us how much the user likes these two genres (acquired through movies.csv).

$$\text{genre signal}(u, i) = \frac{1}{|G_i|} \sum_{g \in G_i} p_{u, g}$$

where G_i is the set of genres movie i belongs to, and $|G_i|$ is the count of those genres.

We also added Decade Bias which learns the rating trends for each release era (split into decades). For example if 1940s movies are rated 0.35 above the average then we can account for this through this bias term (acquired through movies.csv). Similar to previous iterations and how we initialize each bias at 0, we do alternating updates over 20 iterations. Our set of formulas is now written as:

$$b_i = \frac{\sum(r_{ui} - \text{other terms})}{\text{count}_i + \lambda_i}, \quad b_u = \frac{\sum(r_{ui} - \text{other terms})}{\text{count}_u + \lambda_u}, \quad p_{u, g} = \frac{\sum(\text{weighted residuals})}{\text{count}_{u, g} + \lambda_{\text{genre}}}, \quad b_{\text{decade}} = \frac{\sum(\text{residuals})}{\text{count}_{\text{decade}} + \lambda_{\text{decade}}}$$

Thus we now tune two extra regularization terms λ_{genre} and λ_{decade} (grid search across 1, 5, 10, 25, 50). This model improves upon Method 4 because our previous iteration treated all users and items the same, but now it has a better idea of what each user generally gravitates towards and also takes into account whether newer or older movies are more or less popular. The collaborative filtering residual portion is the same as the previous iteration. This method yields a result of...

Item-CF Extended Bias Validation RMSE: 0.8750 ($\lambda_{\text{genre}} = 5, \lambda_{\text{decade}} = 25$)

Our approach is inspired by work on genre-aware collaborative filtering. Al-Safi and Kaleli (2021) demonstrated in their paper that averaging user preferences per genre and building neighbor lists on a per-genre basis allows for accurate recommendations [1]. Also inspired by the Netflix prize approaches presented in lecture.

[1] J. Al-Safi and C. Kaleli, "Item Genre-Based Users Similarity Measure for Recommender Systems," *Appl. Sci.*, vol. 11, no. 13, p. 6108, 2021.

Result Summary Table

Method	Val RMSE
Global Mean	1.1201

User Mean	1.0466
Item Mean	0.9853
Bias Model ($\lambda_u = 5, \lambda_i = 1$)	0.9130
Item-CF ($K = 30$, with bias baseline)	0.8798
Genre-Hybrid CF ($K = 30, \lambda_{genre} = 5, \lambda_{decode} = 25$)	0.8750

Multi-Stage Discussion: Retrieval & Ranking

To view my submission as a two-stage decomposition of retrieval and ranking we can frame it as such:

The goal in the retrieval stage is to get a candidate set of however many items (usually around 100-200 items per user) the user may potentially like. We could use something like the Item Mean method and sort movies by their average rating and take the top N movies, with a fallback to the global mean for movies with no ratings. This is a computationally cheap way for item retrieval (we could maybe even use the simple Bias Model). For the ranking stage, which we will do on the candidate set we got from retrieval, we can use our Genre-Hybrid Collaborative Filtering method which will predict the ratings for each of the candidate items for a user, then sort by the predicted rating and return the ranked list with the highest predicted ratings first.

Stage	Goal	Constraint	Suitable Methods
Retrieval	High recall, approximate scoring	Needs to be easy to compute and fast	User Mean, Item Mean, Bias Model
Ranking	High precision, personalized ordering	Want good accuracy so can have higher compute cost	Item-CF, Genre-Hybrid CF

Ending Analysis

In the `hybridModel.ipynb` notebook we test cold-start items and their validation RMSE (items that have no training ratings) and saw the following results:

Method	Condition	RMSE
Genre-Hybrid CF	cold-start	1.2278
Genre-Hybrid CF	warm-start	0.8750
Global Mean	cold-start	1.3459

The hybrid model still outperforms the global mean model but is doing a lot worse than on items that do have training ratings. This is because cold-start items don't have any neighbors so the collaborative filtering term doesn't have any influence and the model is relying on the extended bias baseline model only. The error we are getting comes from items with few ratings, which cause unreliable item biases with few or no neighbors for the CF correction, and users with few ratings, which causes the CF neighbor search to have less data to work with to find similar movies.

If I had more time I would try to use item metadata similarity for cold items such as genre overlap or year proximity to find pseudo-neighbors so it doesn't just fall back on the baseline model. I would also try using user demographic features and perhaps use latent factor models instead of item-item CF to see if they work better.